

Answers for questions

- 1) When SNR increases, BER decreases exponentially. The higher SNR, the stronger signal power, and the signal levels will be received more accurately.
- 2) No. According to the panel, at SNR = 15 dB, the expected BER is approximately larger than 10^{-5} . At SNR = 11 dB, the BER is approximately 10^{-4} , and BER decreases exponentially.
- 3) As the filter bandwidth decreases, BER increases. The smaller bandwidth is closer to the Nyquist rate (10 GHz for 20 Gbits/s), there will be Intersymbol Interference.
- 4) Signal levels: $[-3/\sqrt{5}, -1/\sqrt{5}, 1/\sqrt{5}, 3/\sqrt{5}]$
Threshold 1 = $(-3/\sqrt{5} + -1/\sqrt{5})/2 = -2/\sqrt{5}$
Threshold 2 = $(-1/\sqrt{5} + 1/\sqrt{5})/2 = 0$
Threshold 3 = $(1/\sqrt{5} + 3/\sqrt{5})/2 = 2/\sqrt{5}$
Thus, the detection thresholds are $[-2/\sqrt{5}, 0, 2/\sqrt{5}]$
- 5) BER will increase. Signal may be disabled to distinguish.

```
%% Lab 2: Digital Transmission Simulations
```

```
clear all; close all; clc;
```

```
fprintf('=== Lab 2: Digital Transmission Simulations - Complete Version  
===\n');
```

```
%% ===== Step 1: Generate random bit vector v  
=====
```

```
fprintf('\n=== Step 1: Generate random bit vector v ===\n');  
num_bits = 100000;  
v = 2*randi([0 1], 1, num_bits) - 1;  
fprintf('Generated %d random bits (1 and -1)\n', num_bits);
```

```
%% ===== Step 2: Generate digital waveform s  
=====
```

```
fprintf('\n=== Step 2: Generate digital waveform s ===\n');  
samples_per_bit = 10;  
bit_period = 50e-12; % 50 ps  
sample_interval = bit_period / samples_per_bit; % 5 ps
```

```
% Generate NRZ waveform using proper method
```

```
s = zeros(1, num_bits * samples_per_bit);  
for i = 1:num_bits  
    start_idx = (i-1)*samples_per_bit + 1;  
    end_idx = i*samples_per_bit;
```

```

        s(start_idx:end_idx) = v(i); % Each bit repeated samples_per_bit times
    end

% Verify implementation
fprintf('Length verification:\n');
fprintf('  Vector v length: %d bits\n', length(v));
fprintf('  Waveform s length: %d samples\n', length(s));
fprintf('  Length ratio: s/v = %d/%d = %.1f\n', length(s), length(v),
length(s)/length(v));

if length(s) == length(v) * samples_per_bit
    fprintf('  ✓ Success: s length is %d times v length\n',
samples_per_bit);
else
    fprintf('  ✗ Error: s length is not %d times v length\n',
samples_per_bit);
end

% Calculate signal power
Ps = mean(s.^2);
fprintf('Signal power Ps = %.4f\n', Ps);

%% ===== Step 3-6: Binary NRZ BER Simulation
=====
fprintf('\n=== Steps 3-6: Binary NRZ BER Simulation ===\n');
SNR_dB = [5, 7, 9, 11, 13, 15, 17];
BER_binary = zeros(size(SNR_dB));

% Define sampling points (middle of each bit)
sample_points = round(samples_per_bit/2) : samples_per_bit : length(s);

for i = 1:length(SNR_dB)
    % Convert SNR from dB to linear scale
    SNR_linear = 10^(SNR_dB(i)/10);

    % Calculate noise variance and standard deviation
    Var_noise = Ps / SNR_linear;
    sigma_noise = sqrt(Var_noise);

    % Add Gaussian noise to the signal
    y = s + sigma_noise * randn(size(s));

    % Sample at middle of each bit
    received_samples = y(sample_points);

```

```

% Make bit decisions (threshold at 0)
decisions = sign(received_samples);

% Calculate BER
errors = sum(decisions ~= v);
BER_binary(i) = errors / num_bits;

fprintf('SNR = %2d dB: Error bits = %d, BER = %.2e\n', ...
        SNR_dB(i), errors, BER_binary(i));
end

% Plot binary NRZ results
figure('Position', [100, 100, 800, 600]);
semilogy(SNR_dB, BER_binary, 'bo-', 'LineWidth', 2, 'MarkerSize', 8);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('Binary NRZ Signal - BER vs SNR');
legend('Simulation Results', 'Location', 'southwest');

%% ===== Step 7: Channel with Low-Pass Filter (using
fft_PolyU/iff_PolyU) =====
fprintf('\n== Step 7: Channel with Low-Pass Filter (using
fft_PolyU/iff_PolyU) ==\n');
bandwidths = [13e9, 10e9, 9e9]; % 13 GHz, 10 GHz, 9 GHz
BER_filtered = zeros(length(bandwidths), length(SNR_dB));

% Create time vector
t = (0:length(s)-1) * sample_interval;

fprintf('System parameters: Sampling frequency = %.0f GHz\n',
1/sample_interval/1e9);

for bw_idx = 1:length(bandwidths)
    bw = bandwidths(bw_idx);
    fprintf('\n--- Filter bandwidth: %.0f GHz ---\n', bw/1e9);

    % Use fft_PolyU to transform to frequency domain
    [omega, spectrum] = fft_PolyU(t, s);

    % Convert angular frequency to Hz and create ideal low-pass filter
    freq_Hz = omega / (2*pi); % Convert rad/s to Hz
    H = double(abs(freq_Hz) <= bw);

```

```

% Apply filter in frequency domain
filtered_spectrum = spectrum .* H;

% Use ifft_PolyU to transform back to time domain
[t_filtered, u] = ifft_PolyU(omega, filtered_spectrum);

% Ensure the signal is real (remove small imaginary parts)
u = real(u);

% Calculate filtered signal power
Ps_filtered = mean(u.^2);
fprintf('Filtered signal power: %.4f\n', Ps_filtered);

for i = 1:length(SNR_dB)
    % Calculate noise parameters
    SNR_linear = 10^(SNR_dB(i)/10);
    Var_noise = Ps_filtered / SNR_linear;
    sigma_noise = sqrt(Var_noise);

    % Add noise to filtered signal
    y = u + sigma_noise * randn(size(u));

    % Sample and make decisions
    received_samples = y(sample_points);
    decisions = sign(received_samples);

    % Calculate BER
    errors = sum(decisions ~= v);
    BER_filtered(bw_idx, i) = errors / num_bits;

    fprintf(' SNR = %2d dB: BER = %.2e\n', SNR_dB(i),
BER_filtered(bw_idx, i));
end
end

% Plot filtered results
figure('Position', [100, 100, 800, 600]);
colors = ['r', 'g', 'm'];
legend_names = {};
for bw_idx = 1:length(bandwidths)
    semilogy(SNR_dB, BER_filtered(bw_idx,:), [colors(bw_idx) 'o-'], ...
'LineWidth', 2, 'MarkerSize', 8);
hold on;

```

```

        legend_names{end+1} = sprintf('%.0f GHz', bandwidths(bw_idx)/1e9);
    end
    grid on;
    xlabel('SNR (dB)');
    ylabel('Bit Error Rate (BER)');
    title('Binary Signals with Different Filter Bandwidths - BER vs SNR');
    legend(legend_names, 'Location', 'southwest');

    %% ===== Step 8: PAM-4 Signal Simulation =====
    fprintf('\n== Step 8: PAM-4 Signal Simulation ==\n');

    % Define PAM-4 levels (normalized for unit average power)
    pam4_levels = [-3/sqrt(5), -1/sqrt(5), 1/sqrt(5), 3/sqrt(5)];
    fprintf('PAM-4 levels: [%.4f, %.4f, %.4f, %.4f]\n', pam4_levels);

    % Generate PAM-4 symbols
    symbol_indices = randi([1, 4], 1, num_bits);
    v_pam4 = pam4_levels(symbol_indices);

    % Verify average power
    Ps_pam4 = mean(v_pam4.^2);
    fprintf('PAM-4 average power: %.4f (target: 1.0000)\n', Ps_pam4);

    % Generate PAM-4 waveform
    s_pam4 = zeros(1, num_bits * samples_per_bit);
    for i = 1:num_bits
        start_idx = (i-1)*samples_per_bit + 1;
        end_idx = i*samples_per_bit;
        s_pam4(start_idx:end_idx) = v_pam4(i);
    end

    % Define detection thresholds (midpoints between levels)
    thresholds = [-2/sqrt(5), 0, 2/sqrt(5)];
    fprintf('PAM-4 detection thresholds: %.4f, %.4f, %.4f\n', thresholds);

    BER_pam4 = zeros(size(SNR_dB));

    for i = 1:length(SNR_dB)
        % Calculate noise parameters
        SNR_linear = 10^(SNR_dB(i)/10);
        Var_noise = Ps_pam4 / SNR_linear;
        sigma_noise = sqrt(Var_noise);
    end

```

```

% Add noise to PAM-4 signal
y = s_pam4 + sigma_noise * randn(size(s_pam4));

% Sample at middle of each symbol
received_samples = y(sample_points);

% PAM-4 symbol decision
decisions_pam4 = zeros(size(received_samples));
for j = 1:length(received_samples)
    if received_samples(j) < thresholds(1)
        decisions_pam4(j) = 1;
    elseif received_samples(j) < thresholds(2)
        decisions_pam4(j) = 2;
    elseif received_samples(j) < thresholds(3)
        decisions_pam4(j) = 3;
    else
        decisions_pam4(j) = 4;
    end
end

% Calculate Symbol Error Rate and Bit Error Rate
symbol_errors = sum(decisions_pam4 ~= symbol_indices);
SER = symbol_errors / num_bits;
BER_pam4(i) = 0.5 * SER; % Assuming Gray coding

fprintf('SNR = %2d dB: Symbol errors = %d, SER = %.2e, BER
= %.2e\n', ...
        SNR_dB(i), symbol_errors, SER, BER_pam4(i));
end

% Plot PAM-4 results
figure('Position', [100, 100, 800, 600]);
semilogy(SNR_dB, BER_pam4, 'ko-', 'LineWidth', 2, 'MarkerSize', 8);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('PAM-4 Signal - BER vs SNR');
legend('PAM-4', 'Location', 'southwest');

%% ===== Comprehensive Comparison =====
% Plot comprehensive comparison
figure('Position', [100, 100, 1000, 600]);
semilogy(SNR_dB, BER_binary, 'bo-', 'LineWidth', 2, 'MarkerSize', 8);
hold on;

```

```

for bw_idx = 1:length(bandwidths)
    semilogy(SNR_dB, BER_filtered(bw_idx,:), [colors(bw_idx) 'o-'], ...
        'LineWidth', 2, 'MarkerSize', 8);
end
semilogy(SNR_dB, BER_pam4, 'ko-', 'LineWidth', 2, 'MarkerSize', 8);
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('Comparison of All Signal Formats - BER vs SNR');
legend_names = {'Binary NRZ'};
for bw_idx = 1:length(bandwidths)
    legend_names{end+1} = sprintf('Binary + %.0fGHz filter',
bandwidths(bw_idx)/1e9);
end
legend_names{end+1} = 'PAM-4';
legend(legend_names, 'Location', 'southwest');

%% ===== Theoretical vs Simulated BER Comparison
%% =====
% Calculate theoretical BER for binary NRZ
SNR_range = 0:0.5:20;
theory_BER = 0.5 * erfc(sqrt(10.^(SNR_range/10)));

figure('Position', [100, 100, 800, 600]);
semilogy(SNR_range, theory_BER, 'r-', 'LineWidth', 2, 'DisplayName',
'Theoretical BER');
hold on;
semilogy(SNR_dB, BER_binary, 'bo-', 'LineWidth', 2, 'MarkerSize', 8,
'DisplayName', 'Simulated BER');
grid on;
xlabel('SNR (dB)');
ylabel('Bit Error Rate (BER)');
title('Theoretical vs Simulated BER - Binary NRZ Signal');
legend('Location', 'southwest');

fprintf('\n=== All steps completed successfully ===\n');
fprintf('Generated 5 figures:\n');
fprintf(' 1. Binary NRZ Signal - BER vs SNR\n');
fprintf(' 2. Binary Signals with Different Filter Bandwidths - BER vs
SNR\n');
fprintf(' 3. PAM-4 Signal - BER vs SNR\n');
fprintf(' 4. Comparison of All Signal Formats - BER vs SNR\n');
fprintf(' 5. Theoretical vs Simulated BER - Binary NRZ Signal\n');

```





